

Task:

(said by Chris)

What we want to do is to implement “matrix multiply followed by subtract max” over two square matrices. You know matrix multiply; what we want is a code that also finds the max value in each row, and subtracts it from each entry in that row.

Here's exactly what I'd like you to do. Start with the "stupid" implementation, where you just do a nested loops matrix multiply, you write out the result, and then you do a row-by-row subtract-max. See how fast it is on a matrix multiply of two matrices both of size M by M , for M in $\{256 * x \text{ for } x \text{ in } 1, 2, 3, \dots, 10\}$. Then try to make it faster. Write a little report. Each time you try something, describe what you did, and if it helped!

Env setup

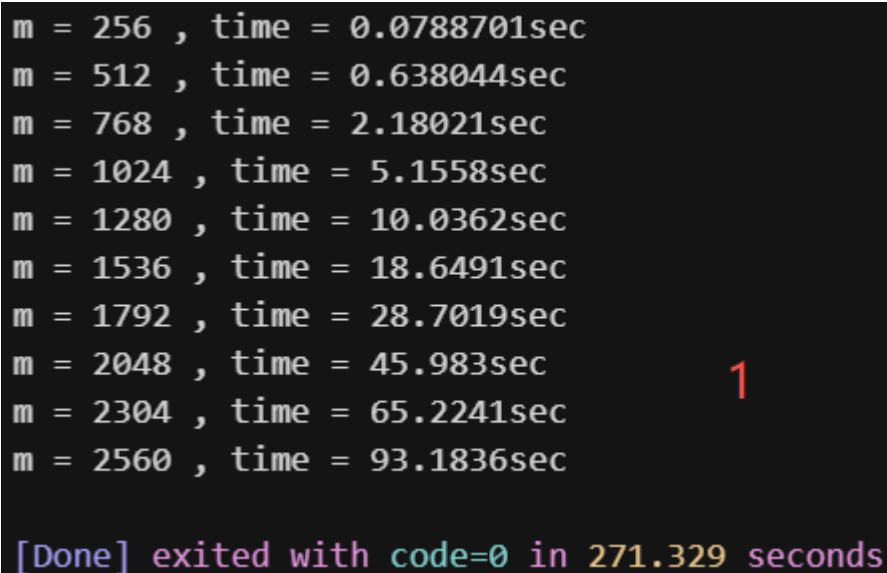
WSL, Git, CPU flag ensure, Makefile, try to run `dgemm_x86[0 - 19]`

Review

[Github repo reference](#)

Play around for opt (C++)

1. naive



```
m = 256 , time = 0.0788701sec
m = 512 , time = 0.638044sec
m = 768 , time = 2.18021sec
m = 1024 , time = 5.1558sec
m = 1280 , time = 10.0362sec
m = 1536 , time = 18.6491sec
m = 1792 , time = 28.7019sec
m = 2048 , time = 45.983sec
m = 2304 , time = 65.2241sec
m = 2560 , time = 93.1836sec

[Done] exited with code=0 in 271.329 seconds
```

comments: quite slow

2. use register

$C(i,j)$ is irrelevant to the innermost loop index k , therefore, one can load it into the register before entering the k -loop to avoid unnecessary memory access.

An array element, it must reside in memory. cij is a local scalar variable, it can be placed in a register.

use pointer

```
m = 256, time = 0.0809329sec
m = 512, time = 0.632992sec
m = 768, time = 2.19695sec
m = 1024, time = 5.19941sec
m = 1280, time = 10.3568sec
m = 1536, time = 18.0607sec
m = 1792, time = 30.5065sec
m = 2048, time = 45.7848sec
m = 2304, time = 65.2288sec
m = 2560, time = 93.1871sec

[Done] exited with code=0 in 272.724 seconds
```

2

comments: theoretically it should be improved. But it seems no use in practice.

3. register blocking

update the whole 2x2 block of C matrix by loading a 2x1 slice of A and an 1x2 slice of B. Then we conduct an outer-product rank-1 update on the 2x2 C block.

```
m = 256, time = 0.0472728sec
m = 512, time = 0.37505sec
m = 768, time = 1.2394sec
m = 1024, time = 2.99279sec
m = 1280, time = 5.9009sec
m = 1536, time = 10.3271sec
m = 1792, time = 14.9509sec
m = 2048, time = 22.965sec
m = 2304, time = 33.3243sec
m = 2560, time = 46.8303sec

[Done] exited with code=0 in 140.494 seconds
```

3

comments: great improvement !

4. register blocking plus

just as same as 3., add more register blocking. (4 * 4)

```

m = 256, time = 0.0251975sec
m = 512, time = 0.204455sec
m = 768, time = 0.683825sec
m = 1024, time = 1.62163sec
m = 1280, time = 3.1502sec
m = 1536, time = 5.49824sec
m = 1792, time = 7.65796sec
m = 2048, time = 12.9531sec
m = 2304, time = 17.2689sec
m = 2560, time = 25.4403sec

[Done] exited with code=0 in 76.294 seconds

```

4

comments: Obviously, it should be improved, because there are more register blockings.

5. SIMD

process data using Single Instruction Multiple Data (SIMD) instructions. SIMD can process 4 float elements at a lane, and improve cache utilization. (__m128)

```

m = 256, time = 0.0205247sec
m = 512, time = 0.158957sec
m = 768, time = 0.536034sec
m = 1024, time = 1.30421sec
m = 1280, time = 2.52174sec
m = 1536, time = 4.32233sec
m = 1792, time = 6.9968sec
m = 2048, time = 8.95456sec
m = 2304, time = 14.918sec
m = 2560, time = 19.1909sec

[Done] exited with code=0 in 60.848 seconds

```

5

comments: because we use the SIMD instructions instead of scalar instructions, we get great great improvement.

6. Kernel 5 plus

update __m128 to __m256, which can process 8 floats in a lane.

Register blocking $4 * 8$

Unroll the loop by 4 folds.

Using __mm256_fmadd_ps, instead of mul + add

```
m = 256, time = 0.00937359sec
m = 512, time = 0.0768506sec
m = 768, time = 0.256747sec
m = 1024, time = 0.616047sec
m = 1280, time = 1.25069sec
m = 1536, time = 2.12961sec
m = 1792, time = 3.87942sec
m = 2048, time = 5.968sec
m = 2304, time = 10.4722sec
m = 2560, time = 14.6317sec
tot: 35 sec
```

6

comments: slightly improves the performance.

ps: should modify 'Code Runner' or use 'bash'

7. Kernel 6 + 8 * 4 blocking

We changed the previous kernel from 4x4 to the current 8x4 so that we obtain a better utilization on all 256-bit YMM registers

```
m = 256, time = 0.00831032sec
m = 512, time = 0.0677775sec
m = 768, time = 0.22744sec
m = 1024, time = 0.541345sec
m = 1280, time = 1.06105sec
m = 1536, time = 1.87088sec
m = 1792, time = 3.02106sec
m = 2048, time = 4.73983sec
m = 2304, time = 7.43703sec
m = 2560, time = 9.37533sec
tot: 25 sec
```

7

comments: we have more blocking, so it should be improved

8. Kernel 7 + cache blocking

At this point, we have `matmul_v8` (cache blocking) + `macro_kernel` (same as kernel 7 implementation) + `scalar_kernel` (edges within block) to deal with the tasks

```
m_blocking: 192
n_blocking: 2048
k_blocking: 384
```

```
Running MatMul ...  
m = 256, time = 0.00930686 sec  
m = 512, time = 0.0754985 sec  
m = 768, time = 0.255378 sec  
m = 1024, time = 0.623481 sec  
m = 1280, time = 1.19075 sec  
m = 1536, time = 2.05084 sec  
m = 1792, time = 3.25804 sec  
m = 2048, time = 4.87892 sec  
m = 2304, time = 6.88253 sec  
m = 2560, time = 8.66519 sec  
tot = 24 sec
```

8

9. Kernel 8 + Packing

Packing the data into continous buffers, eliminating the performance penalty of irregular memory access patterns found in vectors

```
Running MatMul ...  
m = 256, time = 0.00339836 sec  
m = 512, time = 0.0239776 sec  
m = 768, time = 0.0791193 sec  
m = 1024, time = 0.186987 sec  
m = 1280, time = 0.363017 sec  
m = 1536, time = 0.62342 sec  
m = 1792, time = 0.995264 sec  
m = 2048, time = 1.474 sec  
m = 2304, time = 1.25777 sec  
m = 2560, time = 2.89317 sec  
tot = 4 sec
```

9

comments: fast! But code becomes complicated coz it's greatly related to the hardware layer (cache, TBL, memory, pointers stuff)

Some other resources

Easy

Matrix Multiplication

Write a program that multiplies two matrices of 32-bit floating point numbers...

Medium

Sparse Matrix-Vector Multiplication

Implement a GPU program that performs sparse matrix-vector multiplication. Given ...

Medium

General Matrix Multiplication (GEMM)

Implement a basic General Matrix Multiplication (GEMM). Given matrix A ...

Medium

Batched Matrix Multiplication

Implement a batched matrix multiplication in FP32. Given a batch of matrices A of...

Medium

INT8 Quantized MatMul

Implement a quantized matrix multiplication program for 8-bit signed...

Medium

Matrix Power

Implement a GPU program that raises a square matrix A of size $(N \times N)$ to...

Medium

FP16 Batched Matrix Multiplication

Implement a batched matrix multiplication in FP16. Given a batch of matrices A of...

Medium

FP16 Dot Product

Implement a GPU program that computes the dot product of two vectors containing...

```
#include <cuda_runtime.h>
#include <cuda_fp16.h>

__global__ void matmul(const half* A, const half* B, half* C, int M, int
N, int K, float alpha, float beta) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int col = blockIdx.y * blockDim.y + threadIdx.y;
    if (row >= M || col >= N) return;
    float sum = beta * (float)C[row * N + col];
    for (int i = 0; i < K; i++) {
        sum += alpha * (float)A[row * K + i] * (float)B[i * N + col];
    }
    C[row * N + col] = (half)sum;
}

// A, B, and C are device pointers
extern "C" void solve(const half* A, const half* B, half* C, int M, int N,
int K, float alpha, float beta) {
    dim3 threadsPerBlock(16, 16);
    dim3 blockPerGrid((M + threadsPerBlock.x - 1) / threadsPerBlock.x,
                      (N + threadsPerBlock.y - 1) / threadsPerBlock.y);
    matmul<<<blockPerGrid, threadsPerBlock>>>(A, B, C, M, N, K, alpha,
beta);
}
```